UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Master’s Degree in Computer Science

AMM Scheduler:

Design and Implementation of a Runtime Scheduling System for Asymmetric Heterogeneous Computing

Supervisor:

Prof. Marco Danelutto

Candidate:

Simone Stanganini

ACADEMIC YEAR 2024/2025

Contents

1 Introduction **5**

1.1 Contest . . . 5

1.2 Objectives . . . 6

1.3 Thesis Structure . . . 6

2 Tools and Technologies **7**

2.1 Experimental Platform . . . 7

2.1.1 The SoC: Intel Core Ultra 9 185H . . . 8

2.1.2 NVIDIA GeForce RTX 4060 . . . 9

2.2 CUDA Toolkit . . . 9

2.3 SYCL . . . 11

2.4 Openvino . . . 12

2.5 OpenBlass . . . 13

2.6 C/C++ . . . 14

2.7 Cmake . . . 15

3 Logical Design **17**

3.1 Preliminary Phase . . . 18

3.1.1 Studying the Common Patterns . . . 18

3.1.2 Benchmarking . . . 19

3.2 AMM Scheduler . . . 20

4 Implementation **25**

4.1 Project Structure and Compilation . . . 26

4.1.1 Compilation Process . . . 27

4.2 Accellerators Abstraction Layers 33
4.2.1 Cuda 33
4.2.2 OpenVINO 38
4.2.3 SYCL 43
4.2.4 OpenBLASS 47
4.3 AMM Scheduler 50
4.3.1 Thread Initialization 52
4.3.2 Workers 53
4.3.3 Benchmarks Routines 56

5 Results **61**

A Questa è un'appendice **63**

Chapter 1

Introduction

1.1 Contest

Until a few years ago, performance improvements depended mainly on increasing CPU clock speeds, today the approach has changed completely, shifting towards hardware specialization.

Modern computers, from servers to embedded SoCs, don't rely on a single processor anymore, instead they integrate different specific accelerators:

- **CPU**: Optimized for sequential tasks and low latency.
- **GPU**: Available as integrated (iGPU) or discrete (dGPU), in a lot of cases both at the same time, designed for massive parallel computing.
- **NPU** (Neural Processing Unit): A recently introduced unit, specialized in speeding up matrix operations for AI workloads.

This variety of hardware has created a gap with the software, while hardware offers different resources for different workloads, traditional software tends to be static. It usually assigns tasks to a single device ,often just the CPU or GPU, and ignores the other available resources.

The result is an inefficient use of the system. We could get more throughput with the same efficiency, or be more efficient at the same speed, so the current challenge is *orchestration*: we need a system that can dynamically map each task to the right accelerator, using the entire heterogeneous topology of the system.

1.2 Objectives

The main goal of this thesis was to design and implement a scheduler capable of dynamically assigning computational tasks, specifically streams of matrix multiplications, to the different accelerators found in modern architectures.

The work started with an essential preliminary phase, I focused on integrating and studying different compute backends, such as CUDA, SYCL, OpenVINO. This step was useful to create a uniform abstraction layer over the hardware and to verify the technical feasibility of the system.

After this initial phase, the work moved on to the actual development of the scheduler, the core of this project focused on the asymmetric orchestration logic, designed to decide in real-time which resource to use for each specific workload for maximize utilization and efficiency.

Finally, the system was validated with experimental results on a complex hybrid architecture, this setup included an Intel multicore CPU (with integrated GPU and NPU) and a discrete NVIDIA GPU. The tests used to verify the operation were streams of identical matrices, streams of heterogeneous matrices, and splitting larger tasks across multiple accelerators.

1.3 Thesis Structure

The work is organized into the following chapters:

- **Chapter 2**: Introduces the hardware technologies (CPU, GPU, NPU), the software backends used (CUDA, SYCL, OpenVINO, OpenBLAS) and the software stack used for this project (C, C++, Cmake).
- **Chapter 3**: Describes the logical design, divided into two phases: the creation of the benchmarking environment and the subsequent design of the scheduler.
- **Chapter 4**: Details the code implementation, it covers the wrappers for hardware abstraction, the essential data structures for the scheduler (ringbuffer and performance map), and the techniques used for task splitting, as well as the project compilation procedures and details.
- **Chapter 5**: Analyzes the experimental results, validating the effectiveness of the scheduler on different workloads.
- **Chapter 6**: Draws conclusions on the work done and suggests possible future developments.

Chapter 2

Tools and Technologies

This chapter presents the hardware and software resources used to develop and validate the system. The choice to work on a hybrid and highly heterogeneous architecture was made to test the scheduler in a real-world scenario, and in this scenario, devices with different performance characteristics and programming models have to work together.

2.1 Experimental Platform

All development and benchmarking activities were done on a mobile workstation Lenovo Yoga Pro 9i gen 9. This platform represents a modern high-performance "Edge" computing node well, it integrates all the types of accelerators studied into a single physical system: multicore CPU, NPU, integrated GPU, and discrete GPU, and having these components available at the same time allowed to emulate, within a single node, the orchestration issues of heterogeneous distributed systems.

2.1.1 The SoC: Intel Core Ultra 9 185H

The core of the system is the Intel Core Ultra 9 185H processor, based on the "Meteor Lake" architecture [2]. This component marks a significant change from traditional monolithic CPUs. It uses a disaggregated design (tiled architecture) that combines different specialized processing units:

Figure 2.1: Intel Ultra Logo

- **Host CPU**: This cpu is made of 16 hybrid cores, 6 Performance-cores, 8 Efficient-cores, and 2 Low-Power-cores for efficiency. In this project this unit acts both as host and accelerator, it runs the scheduler code and does some computation through the OpenBlass library (2.5).
- **NPU**: The Ultra 9 includes a native Neural Processing Unit. This is a low-power accelerator designed specifically for neural network inference. Its compute acceleration is facilitated by Neural Compute Engines, which consist of hardware acceleration blocks for AI operations like Matrix Multiplication and Convolution, alongside Streaming Hybrid Architecture Vector Engines for general computing tasks [4]. In this project the NPU is used via the OpenVINO 2.4 backend to run Matrix Multiplication.
- **iGPU**: Intel Arc Graphics, integrated into the SoC, this graphics unit based on Xe-LPG architecture [1], offers parallel computing capabilities with direct access to system memory, in this project this accellerator has been used with the SYCL library (2.3) as the backend to leverage the full performance of this accellerator.

2.1.2 NVIDIA GeForce RTX 4060

Figure 2.2: Nvidia Rtx Logo

To complement the Intel SoC the system is equipped with a discrete NVIDIA GeForce RTX 4060 Laptop GPU. This device is based on the Ada Lovelace architecture [8] and acts as the high-throughput accelerator of the system.

Unlike the integrated GPU this card features 3072 CUDA Cores for general parallel processing and 96 Fourth-Generation Tensor Cores. The Tensor Cores are specifically designed to accelerate dense matrix multiplications with 16 or even 8 bit operations.

The device utilizes 8 GB of GDDR6 VRAM with a 128-bit memory interface, this dedicated memory provides high bandwidth but requires explicit data transfer over the PCIe bus.

Within the scheduler this device is managed via the CUDA backend 2.2.

2.2 CUDA Toolkit

Figure 2.3: Nvidia Cuda Logo

To exploit the computational capabilities of the discrete NVIDIA GPU the system utilizes CUDA (Compute Unified Device Architecture). This parallel computing platform and programming model developed by NVIDIA that enables dramatic increases in computing performance by harnessing the power of the GPU. It allows developers to accelerate compute- intensive applications using C, C++, and Fortran, and is widely adopted in fields such as deep learning, scientific computing, and high-performance computing (HPC) [10].

In this project CUDA is the tool chosen to extract maximum throughput from the hardware 2.1.2. The implementation relies on specific libraries and mechanisms to optimize the execution of matrix multiplication streams.

- **cuBLAS and Tensor Cores**: to perform linear algebra operations we adopts cuBLAS (CUDA Basic Linear Algebra Subroutines). This library is the GPU-accelerated version of the standard BLAS specification and provides highly optimized implementations of matrix operations, specifically, cuBLAS allows the utilization of Tensor Cores, specialized hardware units designed to accelerate mixed-precision matrix multiplication (Float16 and Int8). This significantly increases the theoretical throughput compared to standard floating-point units [9].

- **CUDA Streams**: To handle the latency introduced by data transfers between the CPU and the GPU the system utilizes CUDA Streams. A stream is a sequence of operations that execute in issue-order on the GPU, by employing multiple streams it is possible to achieve concurrency: while one stream is transferring data over the PCIe bus another stream can execute a computational kernel [13]. This mechanism, known as "latency hiding", is fundamental for real-time processing and ensures that the GPU compute units remain active as much as possible.

- **NVIDIA Management Library (NVML)**: For the analysis of energy efficiency the project integrates the NVIDIA Management Library (NVML). This serves as a monitoring tool that provides direct access to the GPU hardware sensors, it allows the retrieval of real-time metrics such as power usage (in milliJoule) and temperature without requiring external physical probes [11].

2.3 SYCL

While CUDA provides maximum performance for NVIDIA hardware, it requires a proprietary programming model, and to address the need for portability and to fully exploit the Intel hardware present in the system, the project adopts the SYCL standard through the Intel oneAPI toolkit.

- **The SYCL Standard**:

  Figure 2.4: SYCL Logo

  SYCL is an open, royalty-free standard managed by the Khronos Group. It enables high-performance computing on heterogeneous accelerators (such as CPUs, GPUs, and FPGAs) using standard ISO C++ [15]. As defined in the official specification, SYCL provides an abstraction layer that allows developers to write code for heterogeneous processors using a "single-source" style. This means that host code on the CPU and the device code on the accelerator are written in the same file, utilizing standard C++ templates and lambda functions. This approach eliminates the complexity of managing separate source files for different architectures.

- **Intel oneAPI**:

  Figure 2.5: OneApi Logo

  SYCL requires a specific software development kit (SDK) to be compiled and executed. For this thesis, the Intel oneAPI Base Toolkit was selected [3]. oneAPI is Intel's unified programming model designed to simplify development across diverse architectures, it implements the SYCL standard via the `icpx` compiler, and provides a comprehensive set of libraries optimized for a large set of devices. The choice of this toolkit allows the project to target the available Intel hardware efficiently. Moreover this choice guarantees that the codebase could seamlessly target accellerator from othe vendor like AMD without having to rewrite the code.

2.4 Openvino

Figure 2.6: OpenVino Logo

Developed by Intel, this open-source toolkit is designed to optimize and deploy deep learning models across various types of hardware, from edge devices to cloud servers [5].

The main function of the library is that it acts as a translator, it takes an AI model and converts it into a specific format that the target hardware understands. This is very useful because it allow the program to run on different components like the CPU, NPU or the integrated graphics without needing to rewrite the code for each specific accellerator.

The toolkit automatically handles the low-level details to ensure the task runs as fast as possible in the chosen device. This efficiency is achieved through the model optimizer, as described in the documentation, this tool analyzes the computational operations and simplifies them (for example, by merging multiple mathematical steps into one) before execution.

The result is a hardware-agnostic format called intermediate representation, IR, which allows the runtme to map the operations efficiently to the available resources [5].

In this project OpenVINO was essential for targeting the specific NPU found in the system 2.1.1. Since the NPU cannot run standard C++ code directly, OpenVINO acts as the bridge between the scheduler and the hardware, specifically, it was implemented to execute Matrix Multiplication tasks on the NPU. This allowed the system to offload these calculations to the neural accelerator, taking advantage of its high energy efficiency.

2.5 OpenBlass

Figure 2.7: OpenBlass Logo

To enable efficient processing on the Host CPU, the project integrates OpenBLAS. OpenBLAS is an optimized, open-source implementation of the BLAS (Basic Linear Algebra Subprograms) API.

Historically, the project emerged as a successor to GotoBLAS2, a highly regarded library developed by Kazushige Goto, then after the original development ceased, the OpenBLAS community took over to maintain the code and expand support for modern architectures [16].

The library achieves high performance through a mechanism called runtime processor detection. OpenBLAS first identifies the CPU architecture and automatically selects the most optimized functions for that specific hardware, these functions are often implemented in assembly language to leverage SIMD instructions, such as AVX2 or AVX-512, which allow the processor to handle multiple data points simultaneously. Additionally, the library automatically handles multi-threading to distribute the computational load across all available processor cores[12].

In this project this library allow the Host CPU to be treated as an additional accelerator, so rather than limiting the CPU solely to orchestration tasks, OpenBLAS enables the scheduler to use some of the processor cores as a worker node. More details on this will be provided in the Implementation Chapter 4.

2.6 C/C++

Figure 2.8: C and C++ Logo

The C and C++ programming languages represent two of the most influential and widely used tools in the history of computer science. Known for their performance, versatility, and low-level control, they are employed in a vast range of sectors, from operating systems to high-performance applications.

Below is a brief overview of both languages:

The C Language:

- Developed in the 1970s by Dennis Ritchie, it was one of the first high-level languages in history.
- it is renowned for its simplicity, efficiency, and direct access to memory, making it ideal for writing highly optimized code [6].

The C++ Language:

- C++ is an extension of C developed in the 1980s by Bjarne Stroustrup. Among its key features is the support for Object-Oriented Programming (OOP), which allows for the creation of more structured and modular software compared to procedural C [14].
- It provides a rich Standard Library (STL) and is the industry standard for performance-critical applications.

In the context of this project, C plays a critical architectural role. It was utilized to define the Application Binary Interface (ABI) between the scheduler and the various hardware wrappers. Instead C++ serves as the primary development language. Its adoption was mandatory as it is the native language required by the SDKs of all the utilized frameworks, It is used to implement the complex logic of the scheduler and the data structures necessary for task management. Further details on this will be provided in the Implementation Chapter 4.

2.7 Cmake

Figure 2.9: Cmake Logo

To manage the compilation process of such a complex architecture, the project utilizes CMake (Cross-Platform Make). CMake is an open-source family of tools designed to build, test, and package software. It was originally created by Kitware in 2000 to address the need for a powerful build environment capable of supporting cross-platform development [7].

CMake does not build the software directly, instead it acts as a generator: it reads configuration files named CMakeLists.txt, and generates standard build files for the native toolchain of the operating system, such as Makefiles for Linux or Visual Studio projects for Windows. This abstraction allows developers to describe how the project should be built in a way that is independent of the specific compiler being used.

In this thesis its primary function is to manage the complexity of dynamic linking: since the project depends on large external frameworks (CUDA, OpenVINO, OneAPI, OpenBlass), CMake is configured to locate and link these as shared libraries. This ensures that the executable remains lightweight and that dependencies are resolved correctly at runtime.

Additionally, it enables a modular structure where specific hardware backends can be included or excluded via simple configuration flags.

Chapter 3

Logical Design

This chapter describes the logical structure of the project, the design process was divided into two main phases:

- The first part of the chapter (3.1) presents the Independent Benchmarks. Before developing the centralized scheduler, standalone applications were created for each accelerator. This phase was essential to verify the feasibility of the project and to understand how to effectively control the specific hardware using the selected libraries.

- The second part (3.2) provides a comprehensive overview of the AMM Scheduler. This section describes the high-level operation of all the components of the scheduler, it illustrates the architectural design of the main project and explains how the various logical blocks interact to enable the scheduler's functionality, detailing the specific operational logic that governs the system.

3.1 Preliminary Phase

Before designing the centralized scheduler, the project required a preliminary phase focused on the independent analysis of each software stack. Three standalone applications were developed, targeting the NVIDIA GPU, the Intel iGPU, and the NPU respectively. The primary objective of this phase to gain a deep understanding of the specific libraries (CUDA, SYCL, and OpenVINO) and their interaction with the hardware.

It is important to note that the OpenBLAS backend for the CPU was not included in this preliminary phase. This component was introduced directly during the development of the scheduler (described in the next section 3.2) with a specific goal: to maximize the degree of heterogeneity of the system, by treating the CPU as an additional accelerator rather than just a coordinator. Moreover unlike the GPU and NPU, which required a specific study of their asynchronous drivers and memory models, the CPU integration relies on standard synchronous function calls, consequently, a dedicated standalone analysis deemed unnecessary. The technical details of this integration will be discussed in the implementation chapter (4).

3.1.1 Studying the Common Patterns

A crucial goal of this stage was to gain a deep, hands-on understanding of the specific libraries. Since technologies like CUDA, SYCL, and OpenVINO manage hardware in fundamentally different ways, so a first approach phase was necessary to see how their specific mechanisms works and the drivers interactions.

This distinction is particularly evident in the case of OpenVINO, while CUDA and SYCL follow a traditional *kernel launch* paradigm, the NPU is designed exclusively for deep learning graphs, not for general-purpose linear algebra, consequently, to perform a standard matrix multiplication (GEMM) on this accelerator, a workaround was required, instead of invoking a mathematical function, the operation had to be encapsulated within a synthetic neural network layer, effectively "tricking" the driver into treating a raw calculation that we wanted to accomplish as an inference task (more on this in section **??**).

Despite these architectural differences, this preliminary study revealed a recurring pattern, the execution flow for every accelerator whether running a kernel or a simulated inference relies on four identical logical steps:

- **Context Initialization**: setting up the driver handles like context, queue, and so on.
- **Memory Management**: allocating buffers and managing host-to-device transfers.
- **Kernel Execution**: triggering the asynchronous computation.
- **Synchronization**: waiting for the hardware to complete the task.

Recognizing this common structure for each library was fundamental for the design of the Abstraction Layer used in the scheduler.

3.1.2 Benchmarking

During this phase, benchmarking routines were implemented within each of the standalone applications. However, their purpose was distinct from the final experimental results presented in later chapters, in this context measuring execution time and power consumption served as a development tool to:

- Verify the correctness of the implementation (e.g., ensuring the code was actually running on the accelerator and not falling back to the CPU).
- Analyze the runtime behavior of the drivers, and identify architectural bottlenecks (such as PCIe transfer overheads or compilation latencies).

However, it must be noted that the performance profile observed in these isolated environments proved to be different from the behavior within the full scheduler. The concurrent execution of different workloads and the increased stress on system resources exposed new bottlenecks that were not visible during the standalone tests, consequently, the backend implementations were significantly evolved during the integration phase, and specific structures were introduced to mitigate these latencies. These optimizations and the specific issues encountered under high-load scenarios will be analyzed in detail in the Implementation Chapter (4).

3.2 AMM Scheduler

The core of the project is the AMM Scheduler, that stands for Asymettric Matrix Multiplication Scheduler.

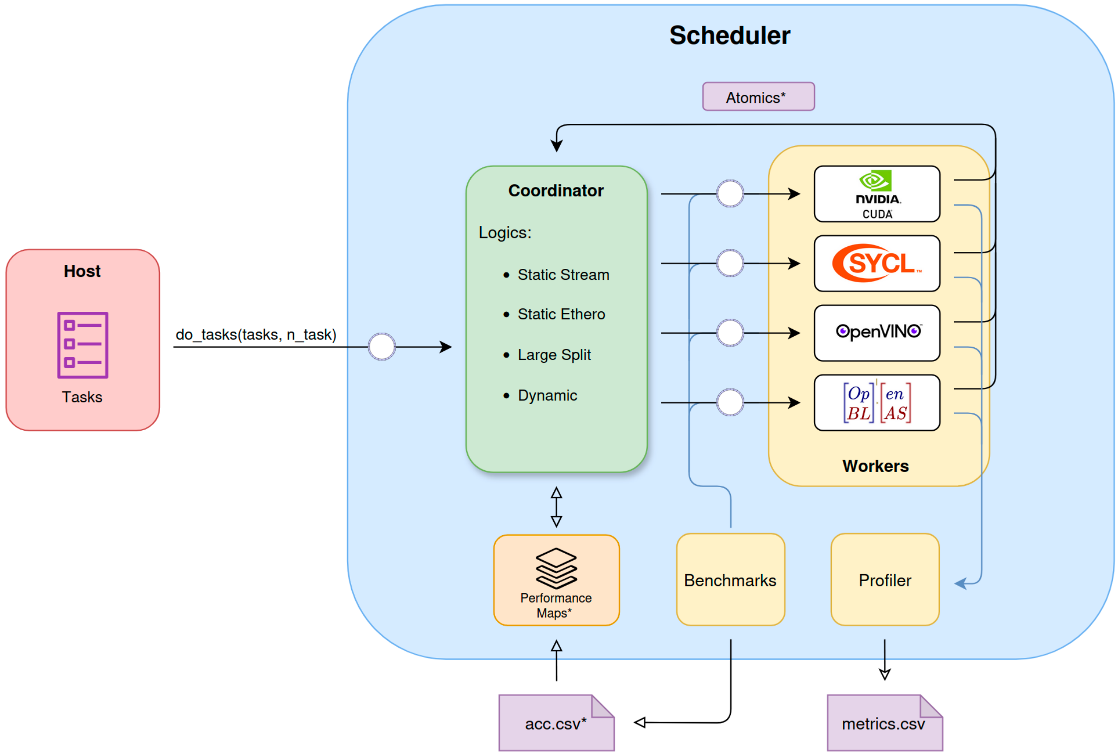

Figure 3.1: AMM Scheduler Components

As illustrated in figure 3.1, the architecture implements the producer and consumer paradigm.

In this model, the host application acts as the producer, generating computational tasks for the scheduler, that functions as the consumer, processing these requests asynchronously. The execution logic follows the farm pattern: a central coordinator retrieves the tasks and distributes them among the available worker nodes. To enable parallel execution, the coordinator and each worker operate as independent threads.

As shown in the diagram, Workers communicate with the Coordinator using atomic variables. This mechanism signals when a Worker is idle or updates the count of completed tasks, this ensures that the scheduler's wait function can correctly determine when all tasks are finished.

Now we will examine each component in the diagram and explain a bit deeper its function:

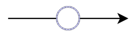

Figure 3.2: Ring Buffer Rappresentation

- **Ring Buffers**: The key to this asynchronous pattern is the ring buffer (rappresented in the figure 3.3 as the arrows marked with a circle), which decouples the Host from the Coordinator, designed ad-hoc for the scheduler, this structure enables extremely fast communication with minimal overhead, using atomic variables instead of locks. As the graph illustrates, this mechanism also serve the communication between the Coordinator and the Workers, minimizing overhead and maximizing speed during task distribution. More on the code implementation in chapter **??**.

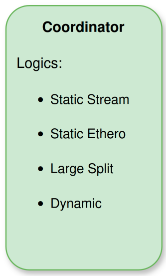

Figure 3.3: Coordinator

- **Coordinator**: The Coordinator acts as the brain of the scheduler, it decides how to dispatch tasks and implements different strategies to demonstrate various resource management techniques.

  It features distinct scheduling logics, based on the type of workload that the host have to compute, the main logics that shape the design of the scheduler are the static one.

  The first logic, Static stream, handles static homogeneous workloads (tasks of identical matrices), by using a Performance Map (a custom data structure containing heuristics and execution time estimates) the coordinator predicts how long a task will take on a specific accelerator and based on this, it performs a static dispatch of the tasks in a batch style manner (see section static partitioning **??**).

  The second logic, Static Hetero, work in a similar way but for static heterogeneous workloads (matrices of different sizes), the coordinator uses the same performance map to calculate the optimal distribution, ensuring the workload is balanced across all accelerators (see section static hetero partitioning **??**).

To evaluate these strategies, we also implemented a dynamic tasks dispatch logic. This approach assigns tasks to the first available Worker (in idle state) and serves as a baseline to verify the efficiency of our static methods (see section dynamic **??**).

Finally, the Coordinator supports the logic of Task Splitting, if a matrix exceeds the memory capacity of a single accelerator, the logic breaks it down into smaller sub-tasks to be distributed across multiple accellerators (see section large matrix split **??**).

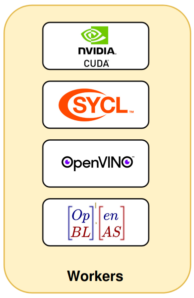

Figure 3.4: workers

- **Workers**: Workers are threads that utilize the abstraction layers defined in the preliminary phase 3.1. Their internal logic is straightforward: they retrieve available tasks and execute them on their assigned accelerator. All low-level details regarding the accelerator libraries are completely hidden from the Worker prospective (see Section **??**).

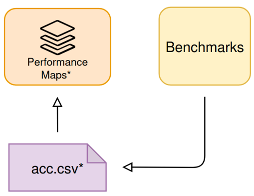

Figure 3.5: benchmark and performance maps rapresentation

- **Benchmarks and Performance Maps**: Internally, the scheduler utilizes a data structure called the Performance Map to determine the execution time of every single task for each individual accelerator. To achieve this, the structure contains heuristics and data loaded from CSV files (specifically one file for each accelerator). These files are created by a benchmark routine developed directly within the scheduler; if the csv for one or more accellerators is missing so that there are no data for the performance maps, the routine activates automatically to generate the necessary information stored in the csv files, it utilizes the accelerator libraries, testing them by executing precise tasks in a specific manner to gather this data as accurately as possible. Furthermore, the Performance

Map is an extremely fast structure: it is equipped with caching and highly optimized internal data structures to minimize overhead on the Coordinator's logic, which is the only component that accesses this structure (see Section **??**).

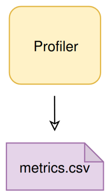

Figure 3.6: profiler

- **Profiler**: To conduct an extensive and precise study of how the scheduler behaves across all workloads, a component called the Profiler was developed directly within the scheduler. It is responsible for profiling all operational strategies and recording all execution times for both the scheduler and the workers. It returns all these metrics in an organized and readable manner in the form of CSV file. These metrics include the power consumption of the entire system and of each individual accelerator, as well as the specific overhead for every single phase of the scheduler's logic and a lot of more information that will be described later (see Section **??**).

Chapter 4

Implementation

In this chapter, after having seen a general explanation of how the scheduler works in the previous sections, we are going to look at and explain all its components in detail.

In the first section, we will explain the structure of the project files and the reason why they are organized this way, and we will also give a view of the entire compilation process of the scheduler.

Subsequently, we will explain each of the abstraction layers used to target the accelerators, and then to conclude, we will see in detail all the components that make the scheduler logic work.

As mentioned in Chapter 3, the code developed during the preliminary phase is not analyzed in this section. This avoids repetition, as the evolved version of this code, which is integrated into the scheduler, will be explained in detail later. However, the original code is available in Appendix **??**.

4.1 Project Structure and Compilation

The figure below provides a view of the project's file and folder structure:

Project Tree

```
|-- bin/
|-- build/
|-- cuda/
|    |-- runner.cu
|    |-- cuda_wrapper.h
|    |-- CMakeLists.txt
|-- sycl/
|    |-- runner.cpp
|    |-- sycl_wrapper.h
|    |-- CMakeLists.txt
|-- openvino/
|    |-- runner.cpp
|    |-- ov_wrapper.h
|    |-- CMakeLists.txt
|-- scheduler/
|    |-- main.cpp
|    |-- scheduler.hpp
|    |-- performancemap.hpp
|    |-- sharedbuffer.hpp
|    |-- cpu.hpp
|    |-- profiler.hpp
|    |-- tasks.hpp
|    |-- tests.hpp
|    |-- config.hpp
|    |-- CMakeLists.txt
|-- CMakeLists.txt
|-- make_run.sh
|-- merge_clangd.py
|-- setup.txt
```

Figure 4.1: Project Dir Tree

The codebase of this project was designed with a specific goal: modularity. Since the project works with very different hardware architectures, writing all the code in a single environment is not feasible. Each accelerator requires its own specific compiler and libraries: NVIDIA GPUs need the nvcc compiler, Intel GPUs need the icpx compiler for SYCL, and the NPU relies on OpenVINO libraries.

To solve this compatibility issue, the architecture was decomposed into independent dynamic libraries. Each hardware backend (CUDA, SYCL, OpenVINO) is compiled into a separate shared object file (.so), and each of these libraries expose a standard C ABI (Application Binary Interface). This design is mandatory because the C ABI acts as a universal language that allows the main scheduler to communicate with the accelerators without needing to understand their specific C++ implementations or complex driver dependencies, moreover the real problem was with the version of the compilers utilized with each specific library, so the standard approach is to unify all the interfaces with the help of the C language.

As shown in Figure 4.1, the project includes a dedicated folder for each library (`sycl/`, `openvino/`, and `cuda/`) containing their specific implementations, finally, the `scheduler/` directory containing all the internal dependencies used by the scheduler logic, which will be explained in section **??**.

4.1.1 Compilation Process

The tool CMake was used in this project to manage this complex build process, the build is not performed all at once.

Below is the CMakeLists.txt file in the root directory:

```
cmake_minimum_required(VERSION 3.15)
project(amm_scheduler)

include(ExternalProject)

set(LIB_OUTPUT_DIR ${CMAKE_SOURCE_DIR}/bin)
file(MAKE_DIRECTORY ${LIB_OUTPUT_DIR})

# CUDA SYCL OPENVINO
option(USE_CUDA "CUDA ON" OFF)
option(USE_SYCL "SYCL ON" OFF)
option(USE_OPENVINO "OpenVINO ON" OFF)
option(USE_OPENBLAS "OpenBLAS ON" OFF)
option(USE_OPENCV "OpenCV ON" OFF)

set(SCHEDULER_DEPS "") # depends variable

set(OPT_FLAGS "-O2 -DNDEBUG")

# CUDA
```

```
if(USE_CUDA)
    message(STATUS "CUDA Backend ON")
    ExternalProject_Add(build_cuda
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/cuda
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            -DCMAKE_CUDA_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )

    list(APPEND SCHEDULER_DEPS build_cuda)
else()
    message(STATUS "CUDA Backend OFF")
endif()

# SYCL
if(USE_SYCL)
    message(STATUS "SYCL Backend: ON")
    ExternalProject_Add(build_sycl
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/sycl
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_COMPILER=icpx
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )
    list(APPEND SCHEDULER_DEPS build_sycl)
else()
    message(STATUS "SYCL Backend: OFF")
endif()

# OPENVINO
if(USE_OPENVINO)
    message(STATUS "OPENVINO Backend: ON")
    ExternalProject_Add(build_openvino
        SOURCE_DIR ${CMAKE_SOURCE_DIR}/openvino
        CMAKE_ARGS
            -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
            -DCMAKE_BUILD_TYPE=Release
            -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
            -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}
            #BUILD_ALWAYS 1
    )
    list(APPEND SCHEDULER_DEPS build_openvino)
else()
    message(STATUS "OPENVINO Backend: OFF")
endif()
```

```
# scheduler
ExternalProject_Add(build_scheduler
    SOURCE_DIR ${CMAKE_SOURCE_DIR}/scheduler
    CMAKE_ARGS
        -DCMAKE_INSTALL_PREFIX=${LIB_OUTPUT_DIR}
        -DCMAKE_BUILD_TYPE=Release
        -DLIBS_DIR=${LIB_OUTPUT_DIR}
        -DCMAKE_CXX_FLAGS_RELEASE=${OPT_FLAGS}

        -DCUDA_SRC_DIR=${CMAKE_SOURCE_DIR}/cuda
        -DSYCL_SRC_DIR=${CMAKE_SOURCE_DIR}/sycl
        -DOPEN_SRC_DIR=${CMAKE_SOURCE_DIR}/openvino

        -DENABLE_CUDA=${USE_CUDA} # USE_CUDA
        -DENABLE_SYCL=${USE_SYCL} # USE_SYCL
        -DENABLE_OPENVINO=${USE_OPENVINO} # USE_OPENVINO
        -DENABLE_OPENBLAS=${USE_OPENBLAS} # USE_OPENBLAS
        -DENABLE_OPENCV=${USE_OPENCV} # USE_OPENCV

        -DCMAKE_EXPORT_COMPILE_COMMANDS=ON # for clangd
    DEPENDS ${SCHEDULER_DEPS}
    #BUILD_ALWAYS 1
)

add_custom_target(UpdateLSP
    ALL
    COMMAND ${CMAKE_COMMAND} -E echo "update LSP..."
    COMMAND python3 ${CMAKE_SOURCE_DIR}/merge_clangd.py
    COMMENT "merge compile_commands.json for clangd"
    DEPENDS build_scheduler
)
```

Listing 4.1: CMakeLists.txt root directory

As the code in the figure 4.1.1 show, the `ExternalProject_Add` feature of cmake is utilized to keep the compilation steps isolated. The main `CMakeLists.txt` file in the root directory acts as a manager. It does not compile the code directly, instead, it checks the configuration flags (such as `USE_CUDA`, `USE_SYCL` and `USE_OPENVINO`) and triggers a separate build process for each active module. The process is the following:

1. First, CMake enters the accellerators sub-directories, `cuda/`, `sycl/`, `openvino/`, and builds them as standalone projects.
2. Each sub-project uses its specific compiler to generate a dynamic library, e.g., `libcuda_lib.so`, `libsycl_lib.so`).
3. Finally, the `scheduler/` directory is compiled. Because of the standard C ABI, the scheduler does not need to know how the hardware works or which compiler

was used for the accelerators, it simply links against these shared libraries at runtime.

This separation ensures that the specific code for one hardware component does not interfere with the others, for example, the scheduler compiler does not need to handle CUDA Kernels, it only sees standard C functions.

Based on the flags passed to CMake during compilation, specific preprocessor definitions are passed to the C++ code. This allows the program to enable or disable dependencies on specific libraries directly inside the source code.

Consequently, the scheduler logic knows exactly which accelerators are available. This approach makes the system flexible, if a specific accelerator or library is missing on the current machine where the project have to be compiled, it can be disabled simply by using a flag.

Below the following snippet from the scheduler's build configuration shows how the flags trigger the injection of preprocessor definitions and the linking of specific libraries:

```
# CUDA
if(NOT DEFINED ENABLE_CUDA)
    message("[SCHEDULER] CUDA off on scheduler")
    set(ENABLE_CUDA OFF)
endif()

# SYCL
if(NOT DEFINED ENABLE_SYCL)
    message("[SCHEDULER] SYCL off on scheduler")
    set(ENABLE_SYCL OFF)
endif()

# OPENVINO
if(NOT DEFINED ENABLE_OPENVINO)
    set(ENABLE_OPENVINO OFF)
endif()

#OPENBLAS
if(NOT DEFINED ENABLE_OPENBLAS)
    message("[SCHEDULER] OpenBLAS off on scheduler")
    set(ENABLE_OPENBLAS OFF)
endif()
```

Listing 4.2: Part of CMakeLists.txt scheduler directory

The command `target_compile_definitions` passes a macro (e.g., `ENABLE_CUDA`) to the compiler, which the C++ code effectively uses to enable or disable code blocks via `#ifdef` directives. Simultaneously, `target_link_libraries` ensures that the scheduler only links against the dynamic libraries that are actually present.

The compilation process for the accelerator backends follows a standard pattern. As an example, the configuration file for the OpenVINO library is presented below:

```
cmake_minimum_required(VERSION 3.20)
project(OpenVinoPart LANGUAGES CXX)

message("\n--------------- OPENVINO COMPILING \n")

find_package(OpenVINO REQUIRED)

add_library(ov_lib SHARED runner.cpp)

set_target_properties(ov_lib PROPERTIES PUBLIC_HEADER ov_wrapper.h)

# C++17 needed OpenVINO
target_compile_features(ov_lib PRIVATE cxx_std_17)

target_link_libraries(ov_lib PRIVATE openvino::runtime)

install(TARGETS ov_lib
    LIBRARY DESTINATION lib
    PUBLIC_HEADER DESTINATION include
)
```

Listing 4.3: CMakeLists.txt OpenVINO directory

First this script locates the necessary dependencies on the host system using `find_package`, then, it defines a shared library named `ov_lib` using the source file `runner.cpp`. The `SHARED` keyword is fundamental because it instructs the compiler to generate a dynamic library instead of a standard executable.

Finally, the script links the specific runtime (in this case, `openvino::runtime`) and specifies the installation rules. The `install` command ensures that the resulting binary and the public header (`ov_wrapper.h`) are placed in the correct output directories, making them accessible to the main scheduler.

The CUDA and SYCL modules utilize an identical structure, differing only in the specific compilers and libraries linked.

Finally, when the compilation finishes, all the resulting files are placed in a `bin/` directory. This folder contains the final executable named scheduler, the csv that the scheduler will create at runtime, and all the dynamic libraries it needs to run.

To speed up the development process, a script named `make_run.sh` was created. This script automates the entire workflow that consist of cleaning the bin and build dir, runs CMake with the correct options to enable all accelerators, compiles the project using all available CPU cores, and finally executes the program.

Additionally, the project includes a Python script named `merge_clangd.py`. Since multiple independent builds are used, the LSP of the code editing tools often don't work with these many libraries, this script collects the compilation commands from all the different modules and merges them into a single file, ensuring that code completion and error highlighting work correctly across the entire project. This file will be executed after all the compilation steps are done.

4.2 Accellerators Abstraction Layers

Before diving into the architectural details in the following sections, it is important to clarify the supported data types. The scheduler is fully capable of managing both 32-bit and 16-bit matrix operations, furthermore thanks to the modular and flexible design of the codebase, the architecture is already provisioned to support 8-bit matrices as well. However, as 8-bit operations are not supported by all target accelerators and were not required to achieve the objectives of this thesis, their implementation has not been finalized across all backends.

4.2.1 Cuda

The CUDA abstraction layer is designed to manage the NVIDIA discrete GPU available in the system. As previously discussed, the interaction between the C++ scheduler and the NVIDIA compiler (`nvcc`) is mediated by a C interface.

The header file `cuda_wrapper.h` defines the Application Binary Interface exposed to the scheduler. It declares the necessary functions for initialization, memory cleanup, and the execution of matrix multiplication across different data types, 32-bit floating point, 16-bit floating point, and 8-bit integer.

```
#ifdef __cplusplus
extern "C" {
#endif

    void cuda_init();
    void cuda_free();
    void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N, int K);
    void cuda_gemm_8bit(void* A, void* B, void* C, int M, int N, int K);

#ifdef __cplusplus
}
#endif
```

Listing 4.4: cuda_wrapper.h interface

The implementation of these functions is in the `runner.cu` file, but before analyzing the initialization and execution logic, we will examine the global data structures and helper macros defined in the file, these elements manage the persistent state of the device and ensure error reporting.

```
#define CHECK_CUDA(func) {                \
    cudaError_t e = (func);                \
    if(e != cudaSuccess)                  \
        printf ("%s %d CUDA ERROR: %s\n", __FILE__,  __LINE__,
    cudaGetErrorString(e)); \
}

#define CHECK_CUBLAS(func) {                          \
    cublasStatus_t e = (func);                        \
    if(e != CUBLAS_STATUS_SUCCESS)                    \
        printf ("%s %d CUBLAS ERROR: ", __FILE__, __LINE__); \
}
#define N_STREAM 2

cublasHandle_t handle;
cudaStream_t streams[N_STREAM];
void *d_A[N_STREAM];
void *d_B[N_STREAM];
void *d_C[N_STREAM];
int i = 0; /* stream id*/
```

Listing 4.5: cuda_runner.cu data and helper

To ensure that the execution goes without errors, the code utilizes two macros: `CHECK_CUDA` and `CHECK_CUBLAS`. These are used to wrap every API call to the CUDA runtime and the cuBLAS library, respectively, if an operation fails the macro immediately prints the specific error code along with the file name and line number where the failure occurred.

Regarding the state management, the system relies on CUDA Streams, a macro `N_STREAM` defines the number of parallel execution queues. Streams, enable the GPU to allows the overlap of memory transfers with computation, or the concurrent execution of multiple kernels. To support this concurrency, the device resources are replicated for each stream. The memory pointers (`d_A`, `d_B`, `d_C`) are defined as arrays, meaning that each stream possesses its own independent pre-allocated memory buffers. Finally, a global integer `i` acts as counter, used to cycle through the available streams when dispatching new tasks.

The design focuses on minimizing the overhead during the runtime by performing expensive operations only once during the startup. The `cuda_init` function is responsible for preparing the GPU environment before any task is processed. Its primary goal is to eliminate the latency associated with running the dynamic memory allocation istrunctions. Allocating memory on the GPU using `cudaMalloc` is a slow operation compare to the others, infact if the system were to allocate and free memory for every single matrix multiplication task, the overhead would significantly impact the performance. To solve this, the initialization function adopts a simple pre-allocation strategy. Below the implementation of the `cuda_init` function:

```
void cuda_init() {
    CHECK_CUBLAS(cublasCreate(&handle));

    /* reset stream counter */
    i = 0;

    size_t MAX = (size_t)4096 * 4096 * sizeof(float);

    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaStreamCreate(&streams[j]));
        CHECK_CUDA(cudaMalloc(&d_A[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_B[j], MAX));
        CHECK_CUDA(cudaMalloc(&d_C[j], MAX));
    }
}
```

Listing 4.6: cuda_runner.cu init

At startup, the function first initializes the cuBLAS library handle, which is required to invoke the linear algebra routines. Then, it iterates through `N_STREAM` number of streams (simply 2 stream is enough). For each stream, it creates a CUDA stream object to manage execution queues and allocates three distinct memory buffers on the device (`d_A`, `d_B`, `d_C`). The size of these buffers is calculated to hold the largest matrix dimension supported by the system (MAX size), ensuring that any incoming task fits within the pre-allocated space without requiring reallocation.

It is important to note that, although the accelerator hardware supports dimensions exceeding this limit, a maximum square matrix size of 4096×4096 was established (for each accellerator). This design choice was made to avoid introducing unnecessary architectural complexity, since it wasn't in this thesis objectives to maximize the memory saturation, and this limit provides a sufficient workload to stress-test the scheduler without complicating the memory management layer.

Complementary to the initialization, the **`cuda_free`** function ensures the proper release of all GPU resources when the scheduler shuts down:

```
void cuda_free() {
    for (int j = 0; j < N_STREAM; ++j) {
        CHECK_CUDA(cudaFree(d_A[j]));
        CHECK_CUDA(cudaFree(d_B[j]));
        CHECK_CUDA(cudaFree(d_C[j]));
        CHECK_CUDA(cudaStreamDestroy(streams[j]));
    }
    CHECK_CUBLAS(cublasDestroy(handle));
}
```

Listing 4.7: cuda_runner.cu free

The function iterates through the created streams, freeing the device memory buffers using `cudaFree` and destroying the stream objects with `cudaStreamDestroy`. Finally, it destroys the global cuBLAS handle to release the library resources.

To maintain compatibility between the C ABI and CUDA C++ features, the execution logic is split into two levels, since the public C interface cannot recognize CUDA specific types it relies on generic `void*` pointers.

To handle this the system implements a layer of simple bridge functions, these functions accept the generic pointers and cast them to the correct concrete type (e.g., `float*` or `__half*`), and immediately forward the call to the internal function.

```
void cuda_gemm_32bit(void* A, void* B, void* C, int M, int N,
    int K) {
     cuda_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K
    );
}

void cuda_gemm_16bit(void* A, void* B, void* C, int M, int N,
    int K) {
     cuda_gemm_16bit_p((__half*)A, (__half*)B, (__half*)C, M, N
    , K);
}
```

Listing 4.8: cuda_runner.cu type bridging functions

The actual computational logic is implemented in the private functions, such as `cuda_gemm_32bit_p`. This functions orchestrates the entire lifecycle of a single GPU task.

```
void cuda_gemm_32bit_p(float* A, float* B, float* C, int M,
    int N, int K) {

        cublasSetStream(handle, streams[i]);

        CHECK_CUDA(cudaMemcpyAsync(d_A[i], A, M * K * sizeof(
    float), cudaMemcpyHostToDevice, streams[i]));
        CHECK_CUDA(cudaMemcpyAsync(d_B[i], B, K * N * sizeof(
    float), cudaMemcpyHostToDevice, streams[i]));

        float a = 1.0f, b = 0.0f;

        cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_32F, N,
                 d_A[i], CUDA_R_32F, K,
                 &b,
                 d_C[i], CUDA_R_32F, N,
```

```
                      CUBLAS_COMPUTE_32F,
                      CUBLAS_GEMM_DEFAULT_TENSOR_OP);

        CHECK_CUDA(cudaMemcpyAsync(C, d_C[i], M * N * sizeof(float), cudaMemcpyDeviceToHost, streams[i]));
        CHECK_CUDA(cudaStreamSynchronize(streams[i])); /* blocking for latency metrics */

        i = (i + 1) % N_STREAM; /* change stream for the next call */
}
```

Listing 4.9: cuda_runner.cu 32-bit logic

The function begins by associating the global cuBLAS handle with the current stream (`streams[i]`), this ensures that all subsequent library calls are queued into this specific stream.

The execution begins by asynchronously copying the inputs A and B to the GPU buffers using the `cudaMemcpyAsync`, next, the matrix multiplication is triggered via `cublasGemmEx`, this function is chosen over standard BLAS calls because it offers more flexibility for high-performance computing.

It enables the `CUBLAS_GEMM_DEFAULT_TENSOR_OP` flag, explicitly instructing the GPU to utilize tensor tores whenever is possible (with 16 bit and 8 bit, in more modern GPUs from the Ampere architecture, even 32 bit).

Notably, the input operands are swapped in the function call, this operation ensure the matematical correctness compensating the layout difference between C++ and cuBLAS that are using two different storing method of the matrix, respectively row-major and column-major, with this simple method the function multiply two matrices without requiring an expensive physical transpose

Once computed, the result is copied back to the host. The process concludes with `cudaStreamSynchronize`, which blocks the thread to ensure data validity and accurate timing, followed by an update of the stream index for the next stream.

Below the implementation for the 16-bit cuda gemm:

```
void cuda_gemm_16bit_p(__half* A, __half* B, __half* C, int M, int N, int K) {
    cublasSetStream(handle, streams[i]);

    CHECK_CUDA(cudaMemcpyAsync(d_A[i], A, M * K * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));
    CHECK_CUDA(cudaMemcpyAsync(d_B[i], B, K * N * sizeof(__half), cudaMemcpyHostToDevice, streams[i]));

    float a = 1.0f, b = 0.0f;

```

```
    cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                 N, M, K,
                 &a,
                 d_B[i], CUDA_R_16F, N,
                 d_A[i], CUDA_R_16F, K,
                 &b,
                 d_C[i], CUDA_R_16F, N,
                 CUBLAS_COMPUTE_32F,
                 CUBLAS_GEMM_DEFAULT_TENSOR_OP);

    CHECK_CUDA(cudaMemcpyAsync(C, d_C[i], M * N * sizeof(
   __half), cudaMemcpyDeviceToHost, streams[i]));
    CHECK_CUDA(cudaStreamSynchronize(streams[i]));

    i = (i + 1) % N_STREAM;
}
```

Listing 4.10: cuda_runner.cu 16-bit logic

The logic remains identical to the 32-bit version, but it introduces changes to handle half-precision data. First the pointer types are replaced with the specific `__half` type. Consequently, the `cublasGemmEx` function is configured with the `CUDA_R_16F` flag for the input and output data. However, it is worth noting that the computation type is kept as `CUBLAS_COMPUTE_32F`, this is a standard practice to ensures that while the data storage is compressed to 16-bit, the internal mathematical computation happens in 32-bit to prevent overflows.

The implementation for 8-bit integer matrices follows the exact same architectural pattern, differing only in data types, `int8_t` and flags `CUDA_R_8I`. The code can be found in full in Appendix ??.

4.2.2 OpenVINO

The OpenVINO abstraction layer is designed to target the Intel NPU, just like the CUDA backend, the interaction between the C++ scheduler and the OpenVINO runtime is mediated by a standard C interface to ensure the ABI compatibility.

```
#ifdef __cplusplus
extern "C" {
#endif

    void ov_init();

    void ov_gemm_32bit(void* A, void* B, void* C, int M, int N
   , int K);
    void ov_gemm_16bit(void* A, void* B, void* C, int M, int N
   , int K);
```

```
    void ov_gemm_8bit(void* A, void* B, void* C, int M, int N,
    int K);

    void ov_free();

#ifdef __cplusplus
}
#endif
```

Listing 4.11: ov_wrapper.h interface

The header file `ov_wrapper.h` defines the interface exposed to the scheduler. It's the identical structure used for the other accelerators, declaring functions for initialization, cleanup, and matrix multiplication for 32-bit, 16-bit, and 8-bit data types.

The implementation resides in the `runner.cpp` file. While the function signatures are similar to the CUDA implementation, the internal logic is fundamentally different due to how OpenVINO function, unlike CUDA, which have all pre-compiled kernels, OpenVINO works by building a so call computational graph and compiling it for the specific target device at runtime.

Compiling a model is an expensive and time consuming operation, it cannot be performed for every single task. To solve this, the system implements a simple caching mechanism.

```
ov::Core core;

using namespace std;

/* cache for compiled models */
static std::map<tuple<int, int, int>, ov::InferRequest>
   request_cache_32bit;
static std::map<tuple<int, int, int>, ov::InferRequest>
   request_cache_16bit;
static std::map<tuple<int, int, int>, ov::InferRequest>
   request_cache_8bit;

const size_t MAX_MAP_SIZE = 20;
size_t map_size_32bit=0;
size_t map_size_16bit=0;
size_t map_size_8bit=0;
```

Listing 4.12: ov runner.cpp chaching

The global variable `core` represents OpenVINO runtime, it will be utilized for discovering available devices and compiling the models.

To implement the caching mechanism, the system utilizes standard C++ maps for associating a specific set of matrix dimensions (represented by a tuple (M,N,K)) with a already compiled ov::InferRequest. This means that if the scheduler requests a matrix multiplication with dimensions that have been seen before, the system can retrieve the pre-compiled request immediately, skipping the expensive compilation step for already seen matrices.

Finally, to manage memory usage and prevent the cache from growing indefinitely in case there are many different matrices, a simple eviction policy is enforced using the `MAX_MAP_SIZE` constant and associated counters and if the number of cached models exceeds this limit, the cache is cleared to make room for new entries.

```
void ov_init_p() {
    bool available = false;

    for (auto &d : core.get_available_devices())
        if (d.find("NPU") != string::npos)
            available = true;

    if (!available) {
        cout << "[OPENVINO] NPU not present, aborting \n";
        exit(EXIT_FAILURE); /* shut down something is wrong */
    }
}
```

Listing 4.13: ov runner.cpp init

The init function simply queries the OpenVINO Core to verify the presence of the NPU device. If the hardware is not found, the system aborts to prevent runtime failures during execution.

The core logic of the OpenVINO backend is found in the caching functions, such as `cache_32bit`. This function is responsible for constructing the neural network graph that represents a matrix multiplication.

```
void cache_32bit(tuple<int, int, int> key) {
    if (request_cache_32bit.find(key) == request_cache_32bit.
  end()) {
        map_size_32bit++;
        if (map_size_32bit >= MAX_MAP_SIZE) {
            request_cache_32bit.clear();
            map_size_32bit = 1;
        }
        auto A_ov = std::make_shared<ov::op::v0::Parameter>(ov
  ::element::f32, ov::Shape{(size_t)get<0>(key), (size_t)get
  <2>(key)});
        auto B_ov = std::make_shared<ov::op::v0::Parameter>(ov
  ::element::f32, ov::Shape{(size_t)get<2>(key), (size_t)get
  <1>(key)});

        auto matmul = std::make_shared<ov::op::v0::MatMul>(
```

```
    A_ov, B_ov);
        auto result = std::make_shared<ov::op::v0::Result>(matmul);

        auto model = std::make_shared<ov::Model>(result, ov::ParameterVector{A_ov, B_ov});

        ov::CompiledModel compiled_model = core.compile_model(model, "NPU");
        ov::InferRequest request = compiled_model.create_infer_request();

        request_cache_32bit[key] = request;
    }
}
```

Listing 4.14: ov runner.cpp caching function

The graph construction consist in three steps, first two input nodes are created with the specific shape and precision in this case 32-bit float. Second, a `MatMul` node is instantiated taking these parameters as inputs, defining the GEMM Operation. Finally, the graph is wrapped into an `ov::Model` object and compiled for the NPU device. This step optimizes the graph for the specific hardware architecture, and the resulting request object is stored in the cache map.

Once the model is cached, the execution of the task is handled by the `ov_gemm_32bit_p` function.

```
void ov_gemm_32bit_p(void *A, void *B, void *C, int M, int N, int K){
    auto key = std::make_tuple(M, N, K);
    cache_32bit(key);

    ov::InferRequest& infer_request = request_cache_32bit[key];

    ov::Tensor tensor_A(ov::element::f32, ov::Shape{(size_t)M, (size_t)K}, A);
    ov::Tensor tensor_B(ov::element::f32, ov::Shape{(size_t)K, (size_t)N}, B);
    ov::Tensor tensor_C(ov::element::f32, ov::Shape{(size_t)M, (size_t)N}, C);

    infer_request.set_input_tensor(0, tensor_A);
    infer_request.set_input_tensor(1, tensor_B);
    infer_request.set_output_tensor(0, tensor_C);

    infer_request.infer();
```

```
}
```

Listing 4.15: ov runner.cpp gemm 32 bit

This function first calls the cache function to ensure the request object is available. Then, it wraps the raw pointers provided by the scheduler (A, B, C) into `ov::Tensor` objects. Notably this constructor uses the existing memory pointers without allocating new buffers or copying data (zero-copy). Finally, the tensors are bound to the input/output ports of the model, and the `infer()` method is called to trigger the execution on the NPU.

Finally, the code exposes the public C ABI by wrapping the internal C++ functions. This structure mirrors the approach used in the CUDA backend, ensuring that the scheduler sees a uniform interface across all accelerators.

```
extern "C" {
    void ov_init() {
        ov_init_p();
    }

    void ov_gemm_32bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_32bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_16bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_16bit_p(A, B, C, M, N, K);
    }

    void ov_gemm_8bit(void *A, void *B, void *C, int M, int N, int K){
        ov_gemm_8bit_p(A, B, C, M, N, K);
    }
    void ov_free() {
    }
}
```

Listing 4.16: ov runner.cpp wrappers

A notable difference compared to the CUDA implementation is the `ov_free` function, which is empty. This is because the OpenVINO runtime relies on modern C++ features, specifically RAII and smart pointers, so when the program terminates or the objects go out of scope, the destructors for the `ov::Core` and the cached requests are invoked automatically, releasing the NPU resources explicit cleanup.

4.2.3 SYCL

The SYCL abstraction layer is designed to manage Intel GPUs. In this project, SYCL is utilized in conjunction with the Intel oneAPI Math Kernel Library (oneMKL) to perform high-performance matrix operations.

Just like the previous modules, the interaction with the scheduler is mediated by a pure C interface to ensure ABI compatibility.

```
#ifdef __cplusplus
extern "C" {
#endif

    void ov_init();

    void ov_gemm_32bit(void* A, void* B, void* C, int M, int N
, int K);
    void ov_gemm_16bit(void* A, void* B, void* C, int M, int N
, int K);
    void ov_gemm_8bit(void* A, void* B, void* C, int M, int N,
    int K);

    void ov_free();

#ifdef __cplusplus
}
#endif
```

Listing 4.17: sycl_wrapper.h interface

The header file `sycl_wrapper.h` defines the standard functions for initialization, execution (32-bit and 16-bit), and cleanup.

The implementation logic begins with the initialization phase:

```
void init() {
    try {

        /* persisten streams */
        for(int i=0; i<N_STREAMS; i++) {
            streams.push_back(std::make_unique<SYCLStream>(MAX
));
        }

    } catch (sycl::exception const& e) {
        std::cerr << "[SYCL ERROR] Init: " << e.what() << "\n"
;
        exit(1);
    }
```

```
}
```

Listing 4.18: sycl runner.cpp init

The primary task of the `init` function is to populate the global vector named `streams` with instances of `SYCLStream`. By iterating `N_STREAMS` times, it creates multiple independent execution contexts, allowing the scheduler to submit tasks in a round-robin manner.

Since SYCL is a higher-level C++ abstraction, the resource management is encapsulated in a dedicated struct named `SYCLStream`.

```
#define N_STREAMS 2

using namespace std;

struct SYCLStream {
    sycl::queue q;
    float *d_A_f, *d_B_f, *d_C_f;
    sycl::half *d_A_h, *d_B_h, *d_C_h;

    SYCLStream(size_t max_elements) : q(sycl::gpu_selector_v, sycl::property::queue::in_order()) {
        d_A_f = sycl::malloc_device<float>(max_elements, q);
        d_B_f = sycl::malloc_device<float>(max_elements, q);
        d_C_f = sycl::malloc_device<float>(max_elements, q);

        d_A_h = sycl::malloc_device<sycl::half>(max_elements, q);
        d_B_h = sycl::malloc_device<sycl::half>(max_elements, q);
        d_C_h = sycl::malloc_device<sycl::half>(max_elements, q);

        if (q.get_device().get_info<sycl::info::device::name>() != "Intel(R) Arc(TM) Graphics") {
            std::cout << "Intel GPU NOT PRESENT\n";
            exit(EXIT_FAILURE);
        }
    }

    ~SYCLStream() {
        sycl::free(d_A_f, q); sycl::free(d_B_f, q); sycl::free(d_C_f, q);
        sycl::free(d_A_h, q); sycl::free(d_B_h, q); sycl::free(d_C_h, q);
    }
};

//static unique_ptr<sycl::queue> global_q;
```

```
static std::vector<std::unique_ptr<SYCLStream>> streams;
static int current_stream = 0;
static size_t MAX = (size_t)4096 * 4096;
```

Listing 4.19: sycl runner.cpp global data

The `SYCLStream` struct acts as a container that holds the state necessary for execution.

First, it contains a `sycl::queue`, which is the object used to submit commands to the device. The queue then is initialized with `in_order`, which simplifies the execution by ensuring that commands submitted to this queue are executed sequentially.

In the constructor, the device memory is pre-allocated using `sycl::malloc_device`, following the same optimization strategy used in the CUDA backend to avoid runtime allocation overhead. A specific check is also performed to verify that the code is running on the Intel GPU specifically, if the GPU is not found, the program aborts.

Finally, the struct includes a destructor, so that when the stream object is destroyed, all the memory buffers associated with it are explicitly freed, preventing memory leaks.

The execution logic is implemented in the following functions:

```
void sycl_gemm_32bit_p(float *A, float *B, float *C, int M,
    int N, int K) {
    SYCLStream& s = *streams[current_stream];

    s.q.memcpy(s.d_A_f, A, M * K * sizeof(float));
    s.q.memcpy(s.d_B_f, B, K * N * sizeof(float));

    float alpha = 1.0f, beta = 0.0f;
    oneapi::mkl::blas::row_major::gemm(s.q, oneapi::mkl::
    transpose::nontrans, oneapi::mkl::transpose::nontrans,
              M, N, K, alpha, s.d_A_f, K, s.d_B_f, N, beta, s.
    d_C_f, N);

    s.q.memcpy(C, s.d_C_f, M * N * sizeof(float));
    s.q.wait();

    current_stream = (current_stream + 1) % N_STREAMS;
}

void sycl_gemm_16bit_p(sycl::half *A, sycl::half *B, sycl::
    half *C, int M, int N, int K) {
    SYCLStream& s = *streams[current_stream];

    s.q.memcpy(s.d_A_h, A, M * K * sizeof(sycl::half));
    s.q.memcpy(s.d_B_h, B, K * N * sizeof(sycl::half));

```

```
    sycl::half alpha = 1.0f, beta = 0.0f;
    oneapi::mkl::blas::row_major::gemm(s.q, oneapi::mkl::transpose::nontrans, oneapi::mkl::transpose::nontrans,
                M, N, K, alpha, s.d_A_h, K, s.d_B_h, N, beta, s.d_C_h, N);

    s.q.memcpy(C, s.d_C_h, M * N * sizeof(sycl::half));
    s.q.wait();

    current_stream = (current_stream + 1) % N_STREAMS;
}
```

Listing 4.20: sycl runner.cpp execution

The `sycl_gemm_32bit_p` function orchestrates execution of the tasks, first, it selects the current stream and copies the input data from the host to the pre-allocated device buffers using `memcpy`.

Next, it invokes the matrix multiplication using the oneMKL library, thanks to this library the matrices A and B in this case can be passed directly with the `nontrans` flag, without needing to swap operands.

Finally, the result is copied back to the host, and `s.q.wait()` is called to block the execution until all operations in the queue are complete.

The 16-bit implementation is structurally identical but operates on the `sycl::half` data type. This allows the hardware to utilize half-precision units on the GPU, improving throughput.

```
extern "C" {
    void sycl_init() {
        init();
    }

    void sycl_gemm_32bit(void *A, void *B, void *C, int M, int N, int K) {
        sycl_gemm_32bit_p((float*)A, (float*)B, (float*)C, M, N, K);
    }

    void sycl_gemm_16bit(void *A, void *B, void *C, int M, int N, int K) {
        sycl_gemm_16bit_p((sycl::half*)A, (sycl::half*)B, (sycl::half*)C, M, N, K);
    }

    void sycl_free() {
        streams.clear();
    }
```

```
}
```

Listing 4.21: sycl runner.cpp wrapper

The code concludes with the C-compatible wrappers. These functions simply cast the generic `void*` pointers to the appropriate C++ types (float* or sycl::half*) and forward the call.

The `sycl_free` function simply calls `streams.clear()`. Because the system uses smart pointers (std::unique_ptr) and the SYCLStream struct has a defined destructor, clearing the vector automatically triggers the destruction of all stream objects.

4.2.4 OpenBLASS

Finally, the project includes a backend to target the host CPU. Unlike the other accelerators which required complex compilation and separate dynamic libraries to ensure compatibility, the CPU implementation was introduced directly into the scheduler codebase.

As already said in this thesis, this library was the last addition to the project, and since the scheduler and the OpenBLASS library share the same compiler, there were no compatibility issues to solve, infact, the implementation is located directly in the header file `cpu.hpp` inside the `scheduler/` directory.

The complete implementation is presented below:

```
inline void cpu_init() {
    openblas_set_num_threads(N_CORES);
    omp_set_num_threads(N_CORES);
}

inline void cpu_gemm_32bit(void* A, void* B, void* C, int M,
    int N, int K) {
    float* A_ = (float*) A;
    float* B_ = (float*) B;
    float* C_ = (float*) C;

    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                M, N, K,
                1.0f,
                A_, K,
                B_, N,
                0.0f,
                C_, N);
}

inline void cpu_gemm_16bit(void* A, void* B, void* C, int M,
    int N, int K) {
```

```
    uint16_t* pA = (uint16_t*)A;
    uint16_t* pB = (uint16_t*)B;
    uint16_t* pC = (uint16_t*)C;

    std::vector<float> A_f32(M*K);
    std::vector<float> B_f32(K*N);
    std::vector<float> C_f32(M*N);

    /* large overhead but we need it, not all cpus have 16 bit
    */
    #pragma omp parallel for
    for (int i = 0; i < M * K; ++i) {
        A_f32[i] = half_to_float(pA[i]);
    }

    for (int i = 0; i < K * N; ++i) {
        B_f32[i] = half_to_float(pB[i]);
    }

    cblas_sgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
                M, N, K,
                1.0f,
                A_f32.data(), K,
                B_f32.data(), N,
                0.0f,
                C_f32.data(), N);

    #pragma omp parallel for
    for (int i = 0; i < M * N; ++i) {
        pC[i] = float_to_half(C_f32[i]);
    }
}
```

Listing 4.22: OpenBLAS CPU Implementation

The initialization function, `cpu_init`, is straightforward, it set the number of thread to use using `openblas_set_num_threads` and for OpenMP `omp_set_num_threads`. This ensures that the CPU utilizes all the available physical cores defined by the `N_CORES` macro.

For the 32 bit execution, the function `cpu_gemm_32bit` acts as a direct wrapper around the standard BLAS routine `cblas_sgemm`. It casts the generic pointers to `float*` and invokes the library function `CblasRowMajor`, which handles the input layout without transposition.

The 16-bit implementation (cpu_gemm_16bit), however, requires a workaround. Since the OpenBLAS library does not support half-precision arithmetic, the function performs a manual conversion. The logic is simple: first convert to Float,

1. Convert to float: the input 16 bit matrices are converted to standard 32 bit

floats.

2. Compute: matrix multiplication is performed using the already available 32-bit routine.

3. Convert to half: The result is converted back to 16 bit.

To mitigate the performance cost of these conversions, the loops are parallelized using OpenMP (`#pragma omp parallel for`).

To handle the bitwise conversion between 16 bit and 32 bit formats without introducing dependencies, two standard IEEE 754 compliant functions were included in the project, these are well-known algorithms already widely used in the open-source community:

```
inline float half_to_float(uint16_t h) {
    uint32_t s = (h >> 15) & 0x00000001;
    uint32_t e = (h >> 10) & 0x0000001f;
    uint32_t m = h & 0x000003ff;

    if (e == 0) {
        if (m == 0) {
            uint32_t res = s << 31;
            float f; memcpy(&f, &res, 4); return f;
        } else {
            while (!(m & 0x00000400)) { m <<= 1; e -= 1; }
            e += 1; m &= ~0x00000400;
        }
    } else if (e == 31) {
        if (m == 0) { // Inf
             uint32_t res = (s << 31) | 0x7f800000;
             float f; memcpy(&f, &res, 4); return f;
        } else { // NaN
             uint32_t res = (s << 31) | 0x7f800000 | (m << 13)
  ;
             float f; memcpy(&f, &res, 4); return f;
        }
    }

    e = e + (127 - 15);
    m = m << 13;
    uint32_t res = (s << 31) | (e << 23) | m;
    float f; memcpy(&f, &res, 4); return f;
}
```

Listing 4.23: IEEE 754 Bitwise Conversion Helpers

The `float_to_half` function (available in the Appendix ??) performs the inverse operation.

4.3 AMM Scheduler

The core of this project is the `AMScheduler` class. While the underlying system involves thread management, synchronization, and hardware interactions, the design goal was to encapsulate all this complexity behind a simple interface.

The scheduler is implemented as a C++ class defined in `scheduler.hpp`, it adheres to the **RAII** pattern. This ensures that the lifecycle of the execution environment including worker threads, shared buffers, and hardware contexts is tied to the life-cycle of the scheduler object itself. However, it is important to note that while standard C++ smart pointers are used for the threads and maps, explicit memory management is maintained for performance critical data structures, such as the task arrays and the internal shared buffers allowing the system to minimize overhead and ensuring that the developer has full visibility into the exact behavior of the memory.

The public interface is minimal, consisting of a constructor, a destructor, and two primary methods for task submission and synchronization:

```
class AMScheduler {

public:

    /* Constructor: initializes threads, benchmarks if needed and performancemaps */
    AMScheduler(Logic logic) {
        init_threads();
        init_benchmarks();
        init_maps();
    }

    /* Destructor: stops threads and cleans up resources */
    ~AMScheduler() {
        stop_threads();
    }

    /* Non blocking: pushes tasks to the coordinator */
    void do_tasks(task* tasks, size_t n) {
        pending_tasks += n;
        for (int i=0; i<n; i++) {
            tasks[i].start_time = chrono::high_resolution_clock::now();
            shared_buffers[BT::CORDINATOR]->put(&tasks[i]);
        }
    }

    /* Blocking: waits until all pending tasks are completed */
    void wait();

```

```
    /* Prints execution statistics */
    void print_stats(task* tasks, int num_tasks);

};
```

Listing 4.24: Scheduler Public Interface

The constructor takes a `Logic` parameter, which decide the scheduling strategy to be used. Upon instantiation, the scheduler perform three operations: it initializes the worker threads and their contexts, loads the benchmark data if presents, and prepares the performance maps. The destructor ensures a clean shutdown of the system, it invokes the `stop_threads` function, which signals all active threads to terminate, waits for them to join, and all all the backends clean functins.

The `do_tasks` method serves as the entry point for workload. Before pushing the task into the system, it records the current timestamp into each task. This timestamp is crucial for the profiler to accurately measure the end-to-end latency of each task. Once timestamped, the task pointer is pushed into the coordinator's shared buffer.

Below is an example of how the scheduler is utilized in the main.cpp file:

```
void test_scheduler() {

    /* 1. Instantiate Scheduler (starts threads) */
    AMScheduler scheduler(Logic::DYNAMIC);

    /* 2. Submit work */
    scheduler.do_tasks(task_array, n_tasks);

    /* ...... */

    /* 3. Wait for completion */
    scheduler.wait();

    /* 4. Analyze profiler results */
    scheduler.print_stats(task_array, n_matrix);
}
```

Listing 4.25: Example usage

The usage of the scheduler is extremely simple, the developer only needs to instantiate the class passing the logic and submit the array of tasks. The scheduler handles all the complexity of thread management and hardware dispatching internally.

4.3.1 Thread Initialization

he `init_threads` function is responsible for setting up the environment based on the available hardware:

```
void init_threads() {

    /* from the fastest to the slowest */
    /* init here and not in the threads */
#ifdef ENABLE_CUDA
        init_acc(BT::CUDA);
        bts.push_back(BT::CUDA);
#endif
#ifdef ENABLE_SYCL
            init_acc(BT::SYCL);
            bts.push_back(BT::SYCL);
#endif
#ifdef ENABLE_OPENVINO
            init_acc(BT::OPENVINO);
            bts.push_back(BT::OPENVINO);
#endif
#ifdef ENABLE_OPENBLAS
            init_acc(BT::OPENBLAS);
            bts.push_back(BT::OPENBLAS);
#endif

    /* initialize atomics for dynamic strategy */
    if(strategy == Logic::DYNAMIC) {
        #ifdef SLEEP
        cout("[SCHEDULER] ERROR remove "#define SLEEP" with Dynamic logic")
        exit(FAILUREEXIT_FAILURE);
        #endif
        for (int i=0; i<bts.size(); i++) {
            busy[bts[i]] = new atomic<bool>;
            *busy[bts[i]] = false;
        }
    }

    for (BT i : bts) {
        shared_buffers[i] = make_unique<SharedBuffer>(BUFFER_LENGHT);
        threads[i] = make_unique<thread>(&AMScheduler::worker_thread, this,
                                                        i, shared_buffers[i].get());
    }

    if (bts.size() == 0) {PRINT("[SCHEDULER] ATTENTION, no accellerator\n"); exit(EXIT_FAILURE);}

```

```
    /* coordinator thread */
    shared_buffers[BT::CORDINATOR] = make_unique<SharedBuffer>(BUFFER_LENGHT);
    threads[BT::CORDINATOR] = make_unique<thread>(&AMScheduler::coordinator_thread,this,
                                ref(shared_buffers)); /* ref because the thread function try to copy all the parameters */
}
```

Listing 4.26: Init Thread

This function first checks which accelerators were enabled during compilation using preprocessor macros passed with CMake. For each enabled backend, it calls the specific initialization function for each backend, inside the dynamic libraries, and adds the backend type to the active list `bts`. After it iterates through the active backends to create the worker threads. For each accelerator, a dedicated `SharedBuffer` is instantiated to hold its incoming tasks, and a standard C++ `std::thread` is spawned to run the `worker_thread` function.

Finally, the function initializes the **Coordinator Thread**. This thread is equipped with its own input buffer and is responsible for fetching tasks from the user and dispatching them to the workers (for implementation see section **??**). The reference to the shared buffers map is passed to the coordinator to allow it to push tasks to any worker inside the logics.

4.3.2 Workers

While the Coordinator thread manages the complexity of deciding where to send the data, the worker threads are designed to be extremely lightweight and simple. Their only purpose is to retrieve a task from their buffer, execute it, and signal the completion.

Below the implementation of the worker function:

```
void worker_thread(BT bt, SharedBuffer *buffer) {
    task* task = nullptr;
    PROF(profiler.init_last_work(bt));

    if (strategy == Logic::DYNAMIC) {

        /* Dynamic execution */
        while (threads_keep_running) {

            task = buffer->get();
            if (!task) {continue;}

```

```
            PROF(profiler.start_worker(bt));
            handle_task(bt, task);
            PROF(profiler.end_worker(bt));

            *busy[bt] = false;
            pending_tasks--;
        }

    } else {

        /* Static execution */
        while (threads_keep_running) {

            task = buffer->get();

            /* if there isn't a task we check if we have to shutdown */
            if (!task) {continue;}

            PROF(profiler.start_worker(bt));
            handle_task(bt, task);
            PROF(profiler.end_worker(bt));
            pending_tasks--;
        }

    }
}
```

Listing 4.27: Worker Thread Implementation

The implementation is a continuous loop that retrieves a task from the shared buffer and processes it. The function `buffer->get()` is the entry point. As detailed in the Shared Buffer section (see Section ??), if the macro `SLEEP` is defined, this function blocks the thread and puts it to sleep until a new task arrives. This prevents the worker from consuming CPU cycles while waiting. If `SLEEP` is not defined, the thread spins (busy wait) to react faster.

The behavior is slightly different based on the scheduler strategy:

- **Static:** The worker simply processes its queue as fast as possible without signaling.
- **Dynamic:** The worker notify the Coordinator when it finishes a task. It sets the atomic flag `*busy[bt]` to false so the Coordinator knows this accelerator is ready for a new task.

The `handle_task` function acts as a wrapper, isolating the scheduler from the specific backend implementations offered by the dynamic libraries linked at runtime.

```
/* task handler for the workers, choose the right backend_type
    */
inline void handle_task(BT backend_type, task *task) {
    switch (backend_type) {
    case BT::CUDA:
#ifdef ENABLE_CUDA
        if (task->type == Type::FLOAT)
            cuda_gemm_32bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
        if (task->type == Type::HALF)
            cuda_gemm_16bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
#endif
        break;

    case BT::SYCL:
#ifdef ENABLE_SYCL
        if (task->type == Type::FLOAT)
            sycl_gemm_32bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
        if (task->type == Type::HALF)
            sycl_gemm_16bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
#endif
        break;

    case BT::OPENVINO:
#ifdef ENABLE_OPENVINO
        if (task->type == Type::FLOAT)
            ov_gemm_32bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
        if (task->type == Type::HALF)
            ov_gemm_16bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
#endif
        break;

    case BT::OPENBLAS:
#ifdef ENABLE_OPENBLAS
        if (task->type == Type::FLOAT)
            cpu_gemm_32bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
        if (task->type == Type::HALF)
            cpu_gemm_16bit(task->A,task->B,task->C, task->M,
    task->N, task->K);
#endif
        break;

    default:
        cerr << "[SCHEDULER] ERROR handle task\n";
        exit(EXIT_FAILURE);
    }
```

```

    task->end_time = chrono::high_resolution_clock::now();
}
```

Listing 4.28: Task Handler

This function performs three simple steps, it checks the `backend_type` enum to decide which accelerator to use, then checks if the task requires 32-bit or 16-bit precision and calls the corresponding C-wrapper function from the dynamic libraries.Immediately after the computation finishes, it saves the current time into `task->end_time`, this gives the precise execution time for the metrics.

4.3.3 Benchmarks Routines

Once the threads are running, the coordinator thread needs specific data to make decisions inside the static logics (see section ??). This data comes from the data profiling saved in CSV files create by `init_benchmarks` function and later uploaded inside the `PerformanceMap` structure (see section ??) by the `init_maps` function.

```
/* upload the banchmark files and generate them if not
   presents */
void init_benchmarks() {
    for (BT i : bts) {

        string filename_f;
        string filename_h;

        filename_f = get_benchmark_filename(i, Type::FLOAT);
        filename_h = get_benchmark_filename(i, Type::HALF);

        if (filesystem::exists(filename_f)) {
            PRINT("[SCHEDULER] File: " << filename_f << "
   opened.\n");
        } else {
            PRINT("[SCHEDULER] File: " << filename_f << "
   not present\n");
            benchmark(i, Type::FLOAT ,filename_f);
            PRINT("[SCHEDULER] Benchmark file created\n");
        }

        if (filesystem::exists(filename_h)){
            PRINT("[SCHEDULER] File: " << filename_h << "
   opened.\n");
        } else {
            PRINT("[SCHEDULER] File: " << filename_h << "
   not present\n");
            benchmark(i, Type::HALF ,filename_h);
            PRINT("[SCHEDULER] Benchmark file created\n");
```

```
        }
    }
}

/* create the map for each accellerator */
void init_maps() {
    for (BT i : bts) {
        string filename_f;
        string filename_h;
        filename_f = get_benchmark_filename(i, Type::FLOAT);
        filename_h = get_benchmark_filename(i, Type::HALF);

        if (filesystem::exists(filename_f) && filesystem::exists(filename_h)) {
            bts_map[i] = make_unique<PerformanceMap>(STEP_SIZE, i, filename_f, filename_h);
        } else {
            cerr << "[SCHEDULER] File: " << filename_f << " or " << filename_h << " not found for map init\n";
            exit(EXIT_FAILURE);
        }
    }
}
```

Listing 4.29: Init Thread

The `init_benchmarks` function ensures that the data exists. It iterates through the list of active accelerators and checks for the existence of specific CSV files for both 32 and 16 bit data types. If a file is found, the system proceeds, if a file is missing, the system automatically triggers the benchmarking routine to generate the data from scratch.

Subsequently, again by the constructor, the `init_maps` function is called. Its role is to load the data from the disk into the memory. It reads the CSV files generated in the previous step and populates the `PerformanceMap` objects for each accelerator. This ensures that when the scheduler is running, it can query the expected execution time of a matrix multiplication in constant time O(1) without needing to read from the disk again. If a file is missing at this stage, the system considers it a critical error and shuts down, as the scheduler cannot function without its performance models. If a file is missing at this stage, the system shuts down, as the scheduler cannot function without its performance models.

The accuracy of the scheduler depends entirely on the quality of the data collected during the benchmarking phase (and small tweaks inside performanceMaps). This process is managed by the `benchmark` and `benchmark_acc` functions.

```
/* call benchmark_acc for each test matrix*/
inline void benchmark(BT bt, Type type, string filename) {
```

```
    vector<int> dims;

    for (int d = STEP_SIZE; d <= STEP_SIZE*STEP_TOTAL; d +=
   STEP_SIZE)
        dims.push_back(d);

    for (int m : dims)
        for (int n : dims)
            for (int k : dims)
                benchmark_acc(bt, type, filename, m, n, k);

    PRINT("[SCHEDULER] Benchmark ===\n");
    PRINT("Results saved in bin/csv/" << filename << "\n");
}
```

Listing 4.30: Benchmark Function

The `benchmark` function acts as the driver. It defines the search space for the matrix dimensions (M, N, K) based on a `STEP_SIZE` (defined in the file `config.hpp`). It uses three nested loops to iterate through all possible combinations of dimensions, calling the measurement function for each configuration.

The measurement logic is implemented in `benchmark_acc`:

```
/* benchmark the accellerator and write in the csv*/
inline void benchmark_acc(BT bt, Type type, string filename,
   int M, int N, int K) {
    chrono::high_resolution_clock::time_point start_time;
    chrono::high_resolution_clock::time_point end_time;
    chrono::duration<double, std::milli> duration;

    const int N_RUNS = 4;
    const int N_TASKS = N_RUNS + 2;

    double total_ms = 0.0;
    double jit_ms = 0.0;
    double no_jit_ms = 0.0;

    /* we simulate a thread */
    task* tasks = init_tasks(N_TASKS, M, N, K, type);

    /* warmup | jit detection */
    start_time = chrono::high_resolution_clock::now();
    handle_task(bt, &tasks[0]);
    end_time = chrono::high_resolution_clock::now();
    duration = end_time - start_time;
    jit_ms = duration.count();

    start_time = chrono::high_resolution_clock::now();
    handle_task(bt, &tasks[1]);
```

```
    end_time = chrono::high_resolution_clock::now();
    duration = end_time - start_time;
    jit_ms = jit_ms - duration.count();

    /* runs */
    for (int i = 2; i < N_TASKS; i++) {
        start_time = chrono::high_resolution_clock::now();
        handle_task(bt, &tasks[i]);
        end_time = chrono::high_resolution_clock::now();
        duration = end_time - start_time;
        total_ms += duration.count();
    }

    clean_tasks(tasks, N_TASKS);

    double avg_ms = total_ms/N_RUNS;

    ofstream file(filename, ios::app);

    if (file.is_open()) {
        if (filesystem::file_size(filename) == 0)
            file << "Accelerator,DataType,M,N,K,Avg_Time_ms,Jit_Time_ms\n";

        string acc_str = get_acc_string(bt);
        file << acc_str << "," << type << "," << M << "," << N << "," << K << "," << avg_ms << "," << jit_ms << "\n";

        PRINT("result: " << acc_str << "," << type << "," << M << "," << N << "," << K << "," << avg_ms << "," << jit_ms << "\n");
        file.close();
    } else {
        cerr << "[SCEDULER] ERROR Cannot open file: " << filename << "\n";
    }
}
```

Listing 4.31: Benchmark Accellerator Function

Measuring the performance of accelerators like GPUs or NPUs is not trivial because these frameworks often perform a just in time compilation (JIT). This means that the very first time a specific matrix operation is executed, the driver must compile the kernel for the hardware, resulting in a significantly higher execution time compared to subsequent runs.

To handle this and isolate the overhead, the function uses a specific strategy, it allocates an array of tasks instead of reusing a single task object. This is done to prevent unrealistic CPU cache hits that might occur if the exact same memory pointers were reused constantly.

The measurement proceeds in three steps: The first task is executed. This run includes the time required for data transfer, computation, and any kernel compilation or driver initialization overhead. The second task is executed immediately after. Since the kernel is already compiled, this run represents the execution time without the JIT expenses, then the system calculates the **JIT time** by subtracting the duration of the second run from the duration of the first run.

The remaining tasks are executed in a loop, their durations are summed and divided by the number of runs to obtain a stable average execution time (`Avg_Time_ms`).

Finally, the results including the accelerator name, data type, matrix dimensions, average time, and JIT time are appended to the corresponding CSV file. This distinction between JIT time and execution time is crucial for the scheduler: during the first few iterations of an algorithm, the scheduler must account for the JIT overhead, and all of this complexity is hidden in the `PerformanceMap`.

Chapter 5

Results

Appendix A

Questa è un'appendice

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Bibliography

[1] Intel Corporation. *Intel Arc Graphics Xe-LPG Architecture*, 2023. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2023-2/intel-xe-gpu-architecture.html.

[2] Intel Corporation. *Intel Core Ultra Processors (Series 1)*, 2024. https://www.intel.com/content/www/us/en/products/sku/236849/intel-core-ultra-9-processor-185h-24m-cache-up-to-5-10-ghz/specifications.html.

[3] Intel Corporation. Intel oneapi: A unified programming model, 2024. https://www.intel.com/content/www/us/en/developer/tools/oneapi/overview.html.

[4] Intel Corporation. *Intel's Neural Processing Unit*, 2024.

[5] Intel Corporation. *Openvino Documentation*, 2025. https://docs.openvino.ai/2025/index.htm.

[6] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1988.

[7] Kitware, Inc. *CMake Documentation*, 2025. https://cmake.org/cmake/help/latest/.

[8] Klaus Hinum. *NVIDIA GeForce RTX 4060 Laptop GPU Spec*, 2022. https://www.notebookcheck.net/NVIDIA-GeForce-RTX-4060-Laptop-GPU-Benchmarks-and-Specs.675692.0.html.

[9] NVIDIA Corporation. *Cublas*, 2025. https://docs.nvidia.com/cuda/cublas/index.html.

[10] NVIDIA Corporation. *CUDA C++ Programming Guide*, 2025. https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[11] NVIDIA Corporation. *NVIDIA Management Library (NVML)*, 2026. https://developer.nvidia.com/management-library-nvml.

[12] OpenBLAS Team. Openblas github, 2024. https://github.com/OpenMathLib/OpenBLAS.

[13] Steve Rennich. *CUDA C++ Streams and Concurrency*, 2024. https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf.

[14] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 2013.

[15] The Khronos Group. Sycl overview and specification, 2024. https://www.khronos.org/sycl/.

[16] Zhang Xianyi. Openblas, 2025. http://www.openmathlib.org/OpenBLAS/.